

Qus 1)

The screenshot shows a web browser displaying a Hackerrank problem titled "1. Collect Points". The problem description states that there are n players and m points on a 1-D space. The initial location of all players is given as an array `players` where `player[i]` denotes the location of the i^{th} player. The location of all points is given as another array `point` where `point[i]` denotes the location of the i^{th} point. In one second, a player can either move 1 step left or 1 step right or not move at all. A point is considered collected if any of the n players has visited the point's location. The players can move simultaneously and independently of each other every second.

The task is to find the minimum time (in seconds) required to collect all the points if players act optimally.

Note: Locations of players and points are not necessarily given in sorted order.

Example
 $n = 3$, `players = [3, 6, 7]`
 $m = 4$, `points = [2, 4, 7, 9]`

All the points can be collected in 2 seconds:

- The first player can collect points at [2] in one second.
- The second player can collect points at [4] in 2 seconds.
- The third player can collect points at [7, 9] in 2 seconds, first collecting the point at 7 and then moving to location 9.

It can be proved that we can't collect all the points in less than 2 seconds. Hence the answer is 2.

The right side of the screen shows a Python code editor with the following code:

```
1 > #!/bin/python3...
10
11 #
12 # Complete the 'getMinimumTime' function below.
13 #
14 # The function is expected to return an INTEGER.
15 # The function accepts following parameters:
16 # 1. INTEGER_ARRAY players
17 # 2. INTEGER_ARRAY points
18 #
19
20 def getMinimumTime(players, points):
21     # Write your code here
22
23 > if __name__ == '__main__':...
```

At the bottom, there are buttons for "Test Results", "Custom Input", "Run Code", "Run Tests", and "Submit". The system clock at the bottom right shows 17:31 on 28-10-2022.

hackerrank.com/test/7nnjskbb24q/questions/cn4kc2nqs0a

Unblockit - Access... Google YouTube Seedr - Quick & Easy Applied Roots QuickRef.ME - Quic... Deep Learning | Co... Topics Wise - OneD... Other bookmarks

1h 28m left

Example
 $n = 3$, $players = [3, 6, 7]$
 $m = 4$, $points = [2, 4, 7, 9]$

All the points can be collected in 2 seconds:

- The first player can collect points at [2] in one second.
- The second player can collect points at [4] in 2 seconds.
- The third player can collect points at [7, 9] in 2 seconds, first collecting the point at 7 and then moving to location 9.

It can be proved that we can't collect all the points in less than 2 seconds. Hence the answer is 2.

Function Description
Complete the function `getMinimumTime` in the editor below. The function must return an integer, the minimum time required to collect all the points
`getMinimumTime` has the following parameters:
players: `players[0]...players[n-1]`: an array of integers denoting the locations of **players**
points: `points[0]...points[m-1]`: an array of integers denoting the locations of points

Constraints

- $1 \leq n, m \leq 10^5$
- $1 \leq players[i], points[j] \leq 10^9$

Input Format For Custom Testing
The first line contains an integer, n , that denotes the number of players.
Each line i of the n subsequent lines (where $0 \leq i < n$) contains an integer that describes `players[i]`.
The next line contains an integer, m , that denotes the number of points.

Language: Python 3 Autocomplete Ready

```
1 > #!/bin/python3...
10
11 #
12 # Complete the 'getMinimumTime' function below.
13 #
14 # The function is expected to return an INTEGER.
15 # The function accepts following parameters:
16 # 1. INTEGER_ARRAY players
17 # 2. INTEGER_ARRAY points
18 #
19
20 def getMinimumTime(players, points):
21     # Write your code here
22
23 > if __name__ == '__main__': ...
```

Test Results Custom Input Run Code Run Tests Submit

hackerrank.com/test/7nnjskxb24q/questions/cn4kc2nqs0a

Each line i of the n subsequent lines (where $0 \leq i < n$) contains an integer that describes $players[i]$.
The next line contains an integer, m , that denotes the number of points.

Each line i of the m subsequent lines (where $0 \leq i < m$) contains an integer that describes $points[i]$.

Sample Case 0

Sample Input For Custom Testing

```
2
4
70
6
76
74
2
75
73
```

Sample Output

```
9
```

Explanation

Given $n = 2$, $m = 6$, $players = [4, 70]$, $points = [76, 9, 74, 2, 75, 73]$
All the points can be collected in 9 seconds:

- The first player can collect points at $[2, 9]$ in 9 seconds, first, it will go to location 2 which takes 2 seconds, and from there to location 9 which takes 7 seconds.
- The second player can collect points at $[73, 74, 75, 76]$ in 6 seconds.

It can be proved that we can't collect all the points in less than 9 seconds. Hence the answer is 9.

Sample Case 1

Language: Python 3

```
1 > #!/bin/python3...
10
11 #
12 # Complete the 'getMinimumTime' function below.
13 #
14 # The function is expected to return an INTEGER.
15 # The function accepts following parameters:
16 # 1. INTEGER_ARRAY players
17 # 2. INTEGER_ARRAY points
18 #
19
20 def getMinimumTime(players, points):
21     # Write your code here
22
23 > if __name__ == '__main__':...
```

Test Results Custom Input Run Code Run Tests Submit

hackerrank.com/test/7nnjskxb24q/questions/cn4kc2nqs0a

Sample Output

```
9
```

Explanation

Given $n = 2$, $m = 6$, $players = [4, 70]$, $points = [76, 9, 74, 2, 75, 73]$
All the points can be collected in 9 seconds:

- The first player can collect points at $[2, 9]$ in 9 seconds, first, it will go to location 2 which takes 2 seconds, and from there to location 9 which takes 7 seconds.
- The second player can collect points at $[73, 74, 75, 76]$ in 6 seconds.

It can be proved that we can't collect all the points in less than 9 seconds. Hence the answer is 9.

Sample Case 1

Sample Input For Custom Testing

```
4
1
2
3
4
4
1
2
3
4
```

Sample Output

```
0
```

Explanation

All the points can be collected in 0 seconds as all the players are already at the location of points. Hence the answer is 0.

Language: Python 3

```
1 > #!/bin/python3...
10
11 #
12 # Complete the 'getMinimumTime' function below.
13 #
14 # The function is expected to return an INTEGER.
15 # The function accepts following parameters:
16 # 1. INTEGER_ARRAY players
17 # 2. INTEGER_ARRAY points
18 #
19
20 def getMinimumTime(players, points):
21     # Write your code here
22
23 > if __name__ == '__main__':...
```

Test Results Custom Input Run Code Run Tests Submit

Solution Qus 1)

C++ (Code by Dhiru)

```
bool valid(vector<int> &ply,vector<int> &po,int mid){
    int i=0,j=0,r=0;
    int n=po.size();
    while(j<n && i<ply.size()){
        if(po[j]<=ply[i]){
            if(ply[i]-po[j]>mid){
                return 0;
            }
        }
        else{
            r=ply[i]-po[j];
            j++;
        }
    }
    while(j<n && po[j]<ply[i] ){
        j++;
    }
    while(j<n && mid-(2*r)>=(po[j]-ply[i])){
        j++;
    }
    i++;
    r=0;
}
return j==n;
}
int fun(){
    vector<int> ply{4,70};
    vector<int> po{76,9,74,2,75,7};

    sort(ply.begin(),ply.end(),[](int &a,int &b){
        return a<b;
    });
    sort(po.begin(),po.end(),[](int &a,int &b){
        return a<b;
    });
    int l=0,r=po.back()-po[0]+1,mid=0;
    while(l<r){
        mid=l+r >> 1;
        if(valid(ply,po,mid)){
            r=mid;
        }
        else{
            l=mid+1;
        }
    }
    debug(r)
    return r;
}
```


Qus 2)

1h 27m left

2. Bob Navigates a Maze

Bob and Alice have teamed up on a game show. After winning the first round, they now have access to a maze with hidden gold. If Bob can collect all the gold coins and deliver them to Alice's position, they can split the gold. Bob can move horizontally or vertically as long as he stays in the maze, and the cell is not blocked.

The maze is represented by an $n \times m$ array. Each cell has a value, where 0 is open, 1 is blocked, and 2 is open with a gold coin. Bob starts at the top left in cell in (row, column) = (0, 0). Alice's position is given by (x, y).

Determine the shortest path Bob can follow to collect all gold coins and deliver them to Alice. If Bob can't collect and give all the gold coins, return -1.

Example: maze = $[[0, 2, 1], [1, 2, 0], [1, 0, 0]]$ with Alice at (2, 2) is represented as follows:

	0	1	2
0	0	2	1
1	1	2	0
2	1	0	0

B represents Bob's starting position at (0, 0), and A represents Alice's position (which will also be Bob's final position). As Bob starts at (0, 0), he has two possible paths to collect the gold and deliver to Alice: (0, 0) → (0, 1) → (1, 1) → (2, 1) → (2, 2) and (0, 0) → (0, 1) → (1, 1) → (1, 2) → (2, 2). Both paths have a length of 4 and could represent the shortest path.

Function Description
Complete the function `minMoves` in the editor below. The function must return the integer length of Bob's shortest path, or -1 if it's not possible.

`minMoves` has the following parameter(s):

Language
C++20

```
1 > #include <bits/stdc++.h> ...
10
11 /*
12  * Complete the 'minMoves' function below.
13  *
14  * The function is expected to return an INTEGER.
15  * The function accepts following parameters:
16  * 1. 2D_INTEGER_ARRAY maze
17  * 2. INTEGER x
18  * 3. INTEGER y
19  */
20
21 int minMoves(vector<vector<int>> maze, int x, int y) {
22
23 }
24
25 > int main() ...
```

Test Results

Custom Input

Run Code

Run Tests

Submit

17:32

20-10-2022

1h 27m left

ALL

1

2

3

4

Function Description
Complete the function `minMoves` in the editor below. The function must return the integer length of Bob's shortest path, or -1 if it's not possible.

minMoves has the following parameter(s):
`maze[maze[0][0]...maze[n-1][m-1]]`: a 2D array of integers
`x`: an integer denoting Alice's row coordinate
`y`: an integer denoting Alice's column coordinate

Constraints

- $1 \leq n, m \leq 100$
- $0 \leq$ the number of coins ≤ 10
- $1 \leq x < n$
- $1 \leq y < m$

Input Format For Custom Testing
The first line contains an integer n , the numbers of rows in `maze`.
The second line contains an integer m , the number of columns in `maze`.
Each of the next n lines contains m space-separated integers that describe the cells of each row in `maze`.
The next line contains an integer x .
The next line contains an integer y .

Sample Case 0
Sample Input 0
STDIN
3
Function Parameters
`maze[][]` number of rows $n = 3$

Language
C++20

Autocomplete Ready

1 > #include <bits/stdc++.h>...
10
11 /*
12 * Complete the 'minMoves' function below.
13 *
14 * The function is expected to return an INTEGER.
15 * The function accepts following parameters:
16 * 1. 2D_INTEGER_ARRAY maze
17 * 2. INTEGER x
18 * 3. INTEGER y
19 */
20
21 int minMoves(vector<vector<int>> maze, int x, int y) {
22
23 }
24
25 > int main()...

Test Results

Custom Input

Run Code

Run Tests

Submit

17:32

28-10-2022

Solution Qus 2) Direct Solution Available

Same question: [Here](#)

##PYTHON SOLUTION

```
import itertools
```

```
def solve(matrix,alice):
```

```
    coins = []
```

```
    for i in range(len(matrix)):
```

```
        for j in range(len(matrix[0])):
```

```
            if matrix[i][j]==2:
```

```
                coins.append((i,j))
```

```
dis = []
```

```
for coin in coins:
```

```
    q = [(coin,0)]
```

```
    cdis = {}
```

```
    for i,d in q:
```

```
        #print(i,d)
```

```
        if i not in cdis and -1<i[0]<len(matrix) and -1<i[1]<len(matrix[0]) and matrix[i[0]][i[1]]!=1:
```

```
            cdis[i]=d
```

```
            q.append(((i[0]+1,i[1]),d+1))
```

```
            q.append(((i[0]-1,i[1]),d+1))
```

```
            q.append(((i[0],i[1]+1),d+1))
```

```
            q.append(((i[0],i[1]-1),d+1))
```

```
    dis.append(cdis)
```

```
#print(dis[0],dis[1])
```

```
def getpathlength(cpath,alice,dis):
```

```
    if (0,0) not in dis[cpath[0]]:
```

```
        return float('inf')
```

```
    path_length = dis[cpath[0]][(0,0)]
```

```
    #print(path_length)
```

```
    for i in range(len(cpath)-1):
```

```
        if coins[cpath[i+1]] not in dis[cpath[i]]:
```

```
            return float('inf')
```

```
            path_length+=dis[cpath[i]][coins[cpath[i+1]]]
```

```
    if alice not in dis[cpath[-1]]:
```

```
        return float('inf')
```

```
    return path_length+dis[cpath[-1]][alice]
```

```
ans = float('inf')
```

```
for cpath in itertools.permutations(range(len(coins))):
```

```
    ans = min(ans,getpathlength(cpath,alice,dis))
```



```
if ans==float('inf'):
    return -1
else:
    return ans
```

Qus 3)

Email - Vinayak Doifode Keshav

HackerRank Question - Paths in a Warehouse

hackerrank.com/test/7nnjskxb24q/questions/52352bfff92eb1

Unblockit - Access... Google YouTube Seedr - Quick & Easy Applied Roots QuickRef.ME - Quic... Deep Learning | Co... Topics Wise - OneD... Other bookmarks

1h 26m left

3. Paths in a Warehouse

A forklift worker moves products from one place to the other in an automotive parts warehouse. There a map in the dashboard that shows, in real time, the open and blocked sections inside the warehouse. The map is displayed as an $n \times m$ matrix of 1's and 0's which represent open and blocked sections respectively. A forklift driver always starts at the upper left corner of the map at $warehouse[0][0]$ and tries to reach the bottom right section of the map or $warehouse[n-1][m-1]$. Each movement must be in increasing value along a row or column but not both. Given the warehouse map, determine the number of distinct paths to get from $warehouse[0][0]$ to $warehouse[n-1][m-1]$. The number may be large, so return the value modulo (10^9+7) .

Example
 $warehouse = [1, 1, 0, 1], [1, 1, 1, 1]$

The matrix below is drawn from the *warehouse* array showing open and blocked sections of the warehouse. 1 indicates an open section and 0 indicates a blocked section. It is only possible to travel through open sections, so no path can go through the section at $(0, 2)$.

Possible Paths

	0	1	2	3
0	1	1	0	1
1	1	1	1	1

	0	1	2	3
0	1	1	0	1
1	1	1	1	1

Language C++20 Autocomplete Ready

```
1 > #include <bits/stdc++.h>...
18
19
20 /*
21  * Complete the 'numPaths' function below.
22  * The function signature is:
23  * The function must return an integer.
24  */
25 Declared in std
26 template <typename _Tp, typename _Alloc =
27 std::allocator<_Tp>>
28 class vector : protected _Vector_base<_Tp, _Alloc> {}
29
30 int numPaths(vector<vector<int>> warehouse) {
31
32 }
33
34 int main() ...
```

Test Results Custom Input Run Code Run Tests Submit

Email - Vinayak Dolfode KeshavHackerRank Question - Paths in a Grid

hackerrank.com/test/7nnjskbb24q/questions/52352bfff92eb1

Unblockit - Access...GoogleYouTubeSeedr - Quick & EasyApplied RootsQuickRef.ME - Quic...Deep Learning | Co...Topics Wise - OneD...Other bookmarks

1h 25m left

The next line contains an integer m , the number of columns in the matrix `warehouse`.
The next n lines each contain a string `warehouse[i]` where $0 \leq i < n$ and $|warehouse[i]| = m$.

Sample Case 0

Sample Input 0

STDIN	Function
3	warehouse[] size n=3 m=4
4	
1 1 1 1	warehouse = [[1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1]]
1 1 1 1	
1 1 1 1	

Sample Output 0

10

Explanation 0

Possible Paths

There are 10 possible paths from `warehouse[0][0]` to `warehouse[2][3]` and 10 modulo $(10^9 + 7) = 10$.

Sample Case 1

LanguageC++20Autocomplete Ready

```
1 > #include <bits/stdc++.h>...
10
11 /*
12  * Complete the 'numPaths' function below.
13  *
14  * The function is expected to return an INTEGER.
15  * The function accepts 2D_INTEGER_ARRAY warehouse as parameter.
16  */
17
18 int numPaths(vector<vector<int>> warehouse) {
19
20 }
21
22 > int main()...
```

Test RegularCustom Input

Run CodeRun TestsSubmit

17:3428-10-2022

1h 26m left

Function Description

Complete the function `numPaths` in the editor below.

`numPaths` has the following parameter(s):

`warehouse[n][m]`: a two dimensional array of integers of n rows and m columns

Returns:

`int`: the number of paths through the matrix, modulo $(10^9 + 7)$.

Constraints

- $1 \leq n, m \leq 1000$
- Each cell in matrix `a` contains either a 0 or a 1.

Input Format for Custom Testing

Input from stdin will be processed as follows and passed to the function.

The first line contains an integer n , the number of rows in the matrix `warehouse`.
The next line contains an integer m , the number of columns in the matrix `warehouse`.
The next n lines each contain a string `warehouse[i]` where $0 \leq i < n$ and $|warehouse[i]| = m$.

Sample Case 0

Sample Input 0

STDIN	Function
3	warehouse size n=3 m=4
4	
1 1 1 1	warehouse = [[1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1]]

Language

Autocomplete Ready

```
1 > #include <bits/stdc++.h>...
10
11
12 /*
13  * Complete the 'numPaths' function below.
14  * The function is expected to return an INTEGER.
15  * The function accepts 2D_INTEGER_ARRAY warehouse as parameter.
16  */
17
18 int numPaths(vector<vector<int>> warehouse) {
19
20 }
21
22 > int main() ...
```

Test Results

Custom Input

Run Code

Run Tests

Submit

Solution Qus 3) Direct Solution Available

Click [Here](#)

##Python Solution

```
from functools import lru_cache
```

```
def solve(matrix):
```

```
    @lru_cache(None)
```

```
    def solver(i,j):
```

```
        if i==len(matrix) or j==len(matrix[0]):
```

```
            return 0
```

```
        if matrix[i][j]==0:
```

```
            return 0
```

```
        if i==len(matrix)-1 and j==len(matrix[0])-1:
```

```
            return 1
```

```
        return solver(i+1,j)+solver(i,j+1)
```

```
    return solver(0,0)
```


Qus 4)

The screenshot shows a web browser displaying a HackerRank problem titled "4. Closest Random Points". The problem description states: "In many real world applications the problem of finding a pair of closest points arises. In the real world, data is usually distributed randomly. Given n points on a plane that are randomly generated with uniform distribution, find the squared shortest distance between pairs of these points."

Example
There are 3 points with x coordinates $x = [0, 1, 2]$ and $y = [0, 1, 4]$. The points have the xy coordinates $(0, 0)$, $(1, 1)$ and $(2, 4)$. The closest points are $(0, 0)$ and $(1, 1)$, and their squared shortest distance is $(1-0)^2 + (1-0)^2 = 2$.

Function Description
Complete the function `closestSquaredDistance` in the editor below.

`closestSquaredDistance` has the following parameter(s):
`int x[n]`: each $x[i]$ denotes the x coordinate of the i^{th} point
`int y[n]`: each $y[i]$ denotes the y coordinate of the i^{th} point

Returns:
`long`: a long integer that denotes the squared shortest distance between the pairs of points

Constraints

- $2 \leq n$
- either $n \leq 1000$ or $n = 10^5$
- values of $x[i]$ and $y[i]$ are randomly generated with uniform distribution from the range $[0, 10^9-1]$

The right side of the image shows a code editor with the following C++ code:

```
1 > #include <bits/stdc++.h> ...
9
10 /*
11  * Complete the 'closestSquaredDistance' function below.
12  *
13  * The function is expected to return a LONG_INTEGER.
14  * The function accepts following parameters:
15  * 1. INTEGER_ARRAY x
16  * 2. INTEGER_ARRAY y
17  */
18
19 long closestSquaredDistance(vector<int> x, vector<int> y) {
20
21 }
22
23 > int main() ...
```

The bottom of the image shows a Windows taskbar with icons for the Start menu, File Explorer, Google Chrome, and PowerPoint. The system clock in the bottom right corner shows the time 17:30 and the date 28-10-2021.

hackerrank.com: test/7mjsknb24q/question/10585594

Unblockit - Access... Google YouTube Seedr - Quick & Easy Applied Roots QuickRef.ME - Quic... Deep Learning | Co... Topics Wise - OneD... Oth

25m left

then, n lines follow.
The i^{th} of them contains the value $x[i]$.
In the next line, there is a single integer n .
Then, n lines follow.
The i^{th} of them contains the value $y[i]$.

Sample Case 0

Sample Input

STDIN	Function
3	→ $x[]$ size $n = 3$
0	→ $x = [0, 10, 15]$
10	
15	
3	→ $y[]$ size $n = 3$
0	→ $y = [0, 10, 20]$
10	
20	

Sample Output

125

Explanation
There are 3 points in the input: $(0,0)$, $(10,10)$, $(15,20)$. The closest squared Euclidean distance among pairs of these points is between $(10,10)$ and $(15,20)$ which is $(15-10)^2 + (20-10)^2 = 125$

Sample Case 1

Sample Case 2

Language: C++20

```
1 > #include <bits/stdc++.h>...
9
10 /*
11  * Complete the 'closestSquaredDistance' function below.
12  *
13  * The function is expected to return a LONG_INTEGER.
14  * The function accepts following parameters:
15  * 1. INTEGER_ARRAY x
16  * 2. INTEGER_ARRAY y
17  */
18
19 long closestSquaredDistance(vector<int> x, vector<int> y)
20 {
21 }
22
23 > int main() ...
```

Test Results Custom Input Run Code Run Tests

Solution Qus 4)

Same Question: [Here](#)

Last Year Question

1h 17m left

1. HackTree

You are given a tree with n nodes. Each node has a value assigned with it. The cost of a path is defined as the summation of all the values assigned to nodes that belong to the path. The root of the tree is node number 1.

Cost of path example

```
graph TD
    1((1)) --- 2((2))
    1 --- 3((3))
    2 --- 4((4))
    3 --- 5((5))
    1 --- 10[10]
    2 --- 15[15]
    3 --- 20[20]
    4 --- 11[11]
    5 --- 31[31]
```

Language: C++

```
1 > #include <bits/stdc++.h>
10
11
12 /* Complete the 'countVerticalPaths' function below.
13 *
14 * The function is expected to return an INTEGER.
15 * The function accepts following parameters:
16 * 1. INTEGER_ARRAY cost
17 * 2. UNWEIGHTED_INTEGER_GRAPH edge
18 * 3. INTEGER k
19 */
20
21 /*
22 * For the unweighted graph, <name>:
23 *
24 * 1. The number of nodes is <name>_nodes.
25 * 2. The number of edges is <name>_edges.
26 * 3. An edge exists between <name>_from[i] and
   <name>_to[i].
27 *
28 */
29
30 int countVerticalPaths(vector<int> cost, int
   edge_nodes, vector<int> edge_from, vector<int>
   edge_to, int k) {
31
32 }
```

Test Custom Run Run Tests Submit

1h 17m left

```
graph TD
    1((1)) --- 6((6))
    1 --- 7((7))
    6 --- 4((4))
    7 --- 5((5))
    1 --- 42[42]
    6 --- 11[11]
    7 --- 11[11]
    4 --- 11[11]
    5 --- 31[31]
```

The cost of the path $6 \rightarrow 5 \rightarrow 3 \rightarrow 1$ in the above tree is $42 + 31 + 20 + 10 = 103$.

A Vertical Path in a tree is the path that is going up towards the root of the tree. It is not necessary for the path to end at the root.

Given a tree with n nodes and an integer k . Find the number of vertical paths such that the (cost of the path) $\% k = 0$ where $\%$ represents the modulo operation.

Note: The modulo operation returns the remainder of a division after one number is divided by another. For example $5 \% 2 = 1$.

Example
 $k = 2$
tree =

Language: C++

```
1 > #include <bits/stdc++.h>
10
11
12 /* Complete the 'countVerticalPaths' function below.
13 *
14 * The function is expected to return an INTEGER.
15 * The function accepts following parameters:
16 * 1. INTEGER_ARRAY cost
17 * 2. UNWEIGHTED_INTEGER_GRAPH edge
18 * 3. INTEGER k
19 */
20
21 /*
22 * For the unweighted graph, <name>:
23 *
24 * 1. The number of nodes is <name>_nodes.
25 * 2. The number of edges is <name>_edges.
26 * 3. An edge exists between <name>_from[i] and
   <name>_to[i].
27 *
28 */
29
30 int countVerticalPaths(vector<int> cost, int
   edge_nodes, vector<int> edge_from, vector<int>
   edge_to, int k) {
31
32 }
```

Test Custom Run Run Tests Submit

Python Solution:

(Code by Kush)

###Hack Tree Python

```
def solve(root,k):
```

```
    ans = 0
```

```
    def traverse(node,csum,counts):
```

```
        if not node:
            return
```

```
        csum+=node.val
```

```
        ans+=counts[csum%k]
```

```
        counts[csum%k]+=1
```

```
        traverse(node.left,csum,counts)
        traverse(node.right,csum,counts)
```

```
    counts = {i:0 for i in range(k)}
    counts[0]=1
```

```
    traverse(root,0,counts)
```

```
    return ans
```